Conteúdo

Projeto 1
Título 1
Objetivo 1
Autor, data, contexto do trabalho 1
Descrição do problema 1
Ferramentas e versões 2
Análise de dados 2
Análise de processos 2
Autenticação e controlo de acesso 2
Criação e gestão de campanhas 3
Gestão de materiais 3
Registo de ações (Fluxo de Campanha) 4
Visualização através de dashboards 4
Descrição de processos 4
Autenticação do User 4
Criação de Campanha 5
Criação e Upload de Material 5
Registo de Ação em FluxoCampanha 5
Visualização de Dashboard 6
Dicionário 6
String connection, instâncias, bases de dados 6
Lista de tabelas, campos e tipos 6
Funcionários 6
Cargos 6
Clientes 6
Campanhas 7
Fluxos de Campanha 7
Materiais 7
Namespaces 7
Classes e métodos 7
CampanhasController 8
ClientesController 8
FluxosCampanhaController 8
FuncionariosController 8
MateriaisController 9
RolesController 9
SeedRoles 9
Outras informações 9
Integração do ASP.NET Core Identity no sistema AdFlow 10

Projeto

Título

AdFlow – Gestor de Campanhas Publicitárias

Objetivo

Desenvolver uma aplicação web que permita planeamento, organização e gestão de campanhas de publicidade online e offline, bem como controlo de acesso conforme o cargo do utilizador.

Autor, data, contexto do trabalho

Rodrigo Fontes, 2026/03/23

O presente trabalho foi desenvolvido no contexto académico como um projeto prático de avaliação, com o objetivo de aplicar os conhecimentos adquiridos durante a formação na área de desenvolvimento de aplicações web.

Descrição do problema

A gestão de campanhas publicitárias envolve diferentes tipos de ações, materiais e profissionais, o que torna o processo difícil quando não existe uma ferramenta centralizada para organização e acompanhamento do trabalho. Em muitos contextos, campanhas são geridas de forma dispersa, dificultando o controlo de prazos, prioridades e responsabilidades por parte dos envolvidos.

Além disso, cada profissional da área de marketing possui funções bem definidas, como criação de materiais gráficos, produção de textos publicitários ou análise e planeamento de campanhas. Quando não há uma separação clara de responsabilidades dentro de um sistema, surgem problemas como acesso indevido a informações, alterações não autorizadas e dificuldade no acompanhamento do progresso das campanhas.

Neste contexto, este projeto propôs o desenvolvimento de uma aplicação web destinada ao planeamento e gestão de campanhas publicitárias, tanto online como offline. A aplicação permite o registo de campanhas associadas a clientes, definindo datas de início e fim, orçamento, nível de prioridade e estado de execução. Cada campanha será criada por um funcionário autorizado e fica disponível apenas para os utilizadores que possuam permissão para atuar sobre o seu tipo.

O sistema permite também a gestão de materiais publicitários, como textos e criativos gráficos. Todas as ações realizadas — como criação de campanhas ou associação de materiais — podem ficar registadas em um fluxo de campanhas, permitindo acompanhar o histórico e a evolução do trabalho.

Por fim, a aplicação disponibiliza dashboards específicos para cada cargo, para facilitar o acompanhamento por gestores e stakeholders e promover uma gestão mais organizada, eficiente e transparente das campanhas publicitárias.

Ferramentas e versões

O projeto foi desenvolvido utilizando as seguintes ferramentas e tecnologias:

- Visual Studio 2026;
- .NET 10;
- ASP.NET Core MVC;
- Entity Framework Core;
- ASP.NET Core Identity (Individual Accounts);
- SQL Server Management Studio (SSMS);
- Bootstrap 5 (para interface).

Análise de dados

A aplicação é baseada em um modelo relacional que organiza a informação em torno das seguintes entidades:

- AspNetUsers (Funcionários) - Utilizadores autenticados do sistema, gestão via ASP.NET Core Identity;
- AspNetRoles (Cargos)– perfis de acesso, como Administrador, Designer, Redator, AnalistaOn e AnalistaOff;
- Cliente – Associa os clientes às suas respectivas campanhas;
- Campanha – Entidade central da aplicação, responsável por armazenar informações sobre campanhas online e offline;
- Material – Representa conteúdos publicitários, podendo ser tipo texto ou criativo gráfico;
- FluxoCampanha – Regista todas as ações realizadas numa campanha, funcionando como histórico de atividades.

As relações são:

- Um cliente pode possuir várias campanhas;
- Uma campanha é criada por um user autenticado;
- Uma campanha pode ter vários registos no FluxoCampanha;
- Cada registo pode estar associado a um Material;
- Um utilizador criar mais de uma campanha, fluxos ou material, conforme suas permissões de acesso.

Análise de processos

A aplicação foi concebida para organizar o fluxo de trabalho de uma equipa de marketing, assegurando controlo de acesso, ações e gestão de campanhas publicitárias online e offline.

O modelo funcional permitiu identificar os seguintes processos:

Autenticação e controlo de acesso

O acesso é realizado através de autenticação com ASP.NET Core Identity (Individual Accounts). Este processo garante:

- Validação de credenciais com segurança;
- Gestão automática de palavras-chave com hash;
- Associação de users a perfis (roles);
- Controlo de acesso baseado em permissões.

Cada user autenticado recebe um role específico:

- Administrador;
- Designer;
- Redator;
- AnalistaOn;
- AnalistaOff.

As permissões determinam:

- Quais páginas podem ser acedidas;
- Quais operações (CRUD) podem ser executadas;

Este processo assegura separação de responsabilidades e proteção de informação.

Criação e gestão de campanhas

A Campanha é a entidade principal do sistema. Este processo envolve:

- Criação de campanhas (apenas users autorizados);
- Definição do tipo (Online ou Offline);
- Associação a um cliente;
- Definição de datas, orçamento e prioridade;
- Associação automática ao user criador da campanha.

O sistema controla:

- Analistas podem gerir campanhas, exceto apagá-las;
- Administrador pode gerir campanhas;
- Outros cargos apenas visualizam conforme permissões.

Gestão de materiais

Os materiais representam os conteúdos que são usados nas campanhas. Existem dois tipos: Este processo inclui:

- Registo do caminho do ficheiro;
- Associação do material a uma campanha;
- Controlo de permissões por cargo.

O objetivo é centralizar as produções e manter organização documental.

Registo de ações (Fluxo de Campanha)

O FluxoCampanha funciona como um mecanismo de rastreabilidade. Sempre que ocorrer uma ação relevante na campanha, deve ser criado um registo contendo:

- Descrição da ação;
- Data;
- User responsável;
- Campanha associada;
- Material associado (quando aplicável).

Este processo permite:

- Acompanhamento cronológico;
- Identificação de responsabilidades;
- Monitorização de evolução.

Visualização através de dashboards

Cada perfil possui um dashboard específico que funciona como uma ferramenta de apoio à decisão e acompanhamento operacional.

Descrição de processos

Autenticação do User

Objetivo: Permitir acesso seguro ao sistema.

Etapas:

a) O user acede à página de Login;
b) Introduz email e palavra-passe;
c) O sistema envia os dados para validação
d) O identity verifica:
   - Existência do user;
   - Correspondência de palavra-passe;
   - Estado da conta.
e) Em caso de sucesso:
   - É criada uma sessão autenticada;
   - O role é identificado;
   - O user é redirecionado para o seu dashboard.
f) Em caso de falha:
   - É apresentada uma mensagem de erro;
   - O acesso é negado.

Criação de Campanha

Objetivo: Permitir criar e planear uma nova campanha.

Etapas:

a) User autorizado acede à área de campanhas;
b) Seleciona "Criar Campanha";
c) Preenche:
   - Nome;
   - Tipo;
   - Cliente;
   - Datas;
   - Orçamento;
   - Prioridade.
d) O sistema valida:
   - Campos obrigatórios;
   - Datas coerentes.
e) O sistema define automaticamente:
   - DataCriacao = Data atual;
   - AspNetUserId = Id do user autenticado.
f) A campanha é guardada na base de dados;

Criação e Upload de Material

Objetivo: Adicionar conteúdo publicitário à campanha.

Etapas:

a) O user acede à área de Materiais;
b) Seleciona tipo (Texto ou Criativo).
c) Digita o caminho do ficheiro;
d) O material fica disponível para ser associado a uma campanha;

Registo de Ação em FluxoCampanha

Objetivo: Registar atualização na campanha.

Etapas:

a) User acede ao detalhe da campanha;
b) Seleciona "Adicionar Fluxo";
c) Introduz descrição
d) Associa material (opcional);
e) O sistema cria registo no FluxoCampanha:
   - Descrição;
   - Data;

- Id do user autenticado;
- Id da campanha;
- Id do material (se existir).

f) O histórico é guardado em ordem cronológica.

Visualização de Dashboard

Objetivo: Apresentar informações resumidas de maneira personalizada por role.

Etapas:

a) User autenticado acede ao dashboard;
b) O sistema identifica o role e mostra o dashboard personalizado.

Dicionário

String connection, instâncias, bases de dados

"DefaultConnection": "Server=62.28.39.135,62444;Database=Proj25H_14_Rodrigo;User Id=efa0125;password=efa0125@;TrustServerCertificate=True;"

Lista de tabelas, campos e tipos

Funcionários

- Nome (string);
- Email (Email);
- NIF (string);
- Endereço (String);
- Cargo (AspNetRoles)

Cargos

- Nome do Cargo (String);

Clientes

- Nome (string);
- NIF (string);
- Endereço (string);
- Telemóvel (string);
- Classe (string);

Campanhas

- Nome Campanha (string);
- Data Criação (data);
- Tipo (enum);
- Data Início (data);
- Data Fim (data);
- Orçamento (double);
- Prioridade (enum);
- Status (enum);
- Cliente (FK);
- Criado Por (FK);

Fluxos de Campanha

- Descrição (string);
- Data Criação (data);
- Campanha (FK);
- Material (FK);
- Funcionário (FK);

Materiais

- Material (string);
- Tipo (enum);
- Caminho do Ficheiro (string);
- Data Criação (data);
- Criado Por (FK);

Namespaces

- AdFlow.Controllers
- AdFlow.Data
- AdFlow.Migrations
- AdFlow.Models
- AdFlow.Models.Enums
- AdFlow.Models.Identity

Classes e métodos

CampanhasController

- CampanhasController(ApplicationDbContext context, UserManager<ApplicationUser> userManager)
- Task<IActionResult> Index(int? clienteId)
- Task<IActionResult> Details(int? id)

- IActionResult Create()
- Task<IActionResult> Create(Campanha campanha)
- Task<IActionResult> Edit(int? id)
- Task<IActionResult> Edit(int id, Campanha campanha)
- Task<IActionResult> Delete(int? id)
- Task<IActionResult> DeleteConfirmed(int id)
- bool CampanhaExists(int id)

ClientesController

- ClientesController(ApplicationDbContext context)
- Task<IActionResult> Index(string classe)
- Task<IActionResult> Details(int? id)
- IActionResult Create()
- Task<IActionResult> Create(Cliente cliente)
- Task<IActionResult> Edit(int? id)
- Task<IActionResult> Edit(int id, Cliente cliente)
- Task<IActionResult> Delete(int? id)
- Task<IActionResult> DeleteConfirmed(int id)
- bool ClienteExists(int id)

FluxosCampanhaController

- FluxosCampanhaController(ApplicationDbContext context, UserManager<ApplicationUser> userManager)
- Task<IActionResult> Index(int? campanhaId)
- Task<IActionResult> Details(int? id)
- IActionResult Create(int campanhaId)
- Task<IActionResult> Create(FluxoCampanha fluxoCampanha)
- Task<IActionResult> Edit(int? id)
- Task<IActionResult> Edit(int id, FluxoCampanha fluxoCampanha)
- Task<IActionResult> Delete(int? id)
- Task<IActionResult> DeleteConfirmed(int id)
- bool FluxoCampanhaExists(int id)

FuncionariosController

- FuncionariosController(ApplicationDbContext context, UserManager<ApplicationUser> userManager, RoleManager<IdentityRole> roleManager)
- Task<IActionResult> Index()
- Task<IActionResult> Edit(string id)
- Task<IActionResult> Edit(string id, ApplicationUser funcionario, string role)

- Task<IActionResult> Delete(string id)
- Task<IActionResult> DeleteConfirmed(string id)
- bool FuncionarioExiste(string id)

MateriaisController

- MateriaisController(ApplicationDbContext context, UserManager<ApplicationUser> userManager)
- Task<IActionResult> Index(TipoMaterial? tipo)
- Task<IActionResult> Details(int? id)
- IActionResult Create()
- Task<IActionResult> Create(Material material)
- Task<IActionResult> Edit(int? id)
- Task<IActionResult> Edit(int id, Material material)
- Task<IActionResult> Delete(int id)
- Task<IActionResult> DeleteConfirmed(int id)
- bool MaterialExists(int id)

RolesController

- RolesController(RoleManager<IdentityRole> roleManager)
- IActionResult Index()
- IActionResult Create()
- Task<IActionResult> Create(string name)
- Task<IActionResult> Edit(string id)
- Task<IActionResult> Edit(string id, IdentityRole role)
- Task<IActionResult> Delete(string id)
- Task<IActionResult> DeleteConfirmed(string id)

SeedRoles

- Task CreateRoles(IServiceProvider serviceProvider)

Outras informações

Integração do ASP.NET Core Identity no sistema AdFlow

No desenvolvimento do sistema **AdFlow**, foi implementado o mecanismo de autenticação e autorização através do **ASP.NET Core Identity**, com o objetivo de garantir a gestão segura de utilizadores, controlo de acessos e definição de permissões por perfil.

Extensão da classe de utilizador

Inicialmente, foi criada uma classe personalizada denominada ApplicationUser, que herda de IdentityUser. Esta abordagem permitiu estender o modelo padrão de utilizador fornecido pelo Identity, possibilitando a inclusão de atributos adicionais relevantes ao contexto da aplicação.

Foram adicionados os seguintes campos:

- Nome
- NIF
- Endereço

Configuração do Identity no Program.cs

A configuração do ASP.NET Core Identity foi realizada no ficheiro Program.cs, onde foram registados os serviços necessários à autenticação e gestão de utilizadores.

Foi associada a classe ApplicationUser ao sistema:

```
builder.Services.AddDefaultIdentity<ApplicationUser>()
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>();
```

A inclusão de .AddRoles<IdentityRole>() foi essencial para permitir a gestão de cargos (roles), possibilitando posteriormente a implementação de controlo de acessos baseado em perfis de utilizador.

Utilização do UserManager

O UserManager<ApplicationUser> foi utilizado como principal serviço para operações relacionadas com utilizadores, incluindo:

- Criação de novos utilizadores
- Associação de dados personalizados no registo
- Autenticação de utilizadores
- Associação de cargos (roles) aos utilizadores

Durante o processo de registo (Register), foi efetuado o preenchimento dos campos adicionais do utilizador, seguido da criação da conta através do método:

```
await _userManager.CreateAsync(user, password);
```

Posteriormente, foi realizada a associação do utilizador a um cargo específico:

```
await _userManager.AddToRoleAsync(user, role);
```

Adicionalmente, o UserManager foi utilizado para obter o utilizador autenticado no sistema, permitindo associar ações (como criação de campanhas ou materiais) ao respetivo responsável.

Utilização do RoleManager

Para a gestão de cargos, foi utilizado o serviço RoleManager<IdentityRole>, responsável por operações relacionadas com os perfis de acesso do sistema.

Com este serviço, foi possível:

- Criar cargos (roles)
- Editar cargos existentes
- Remover cargos
- Verificar a existência de cargos

Foi também desenvolvido um mecanismo de inicialização (seeding) através da classe SeedRoles, garantindo que os cargos essenciais do sistema fossem automaticamente criados na base de dados caso não existissem.

Os cargos definidos foram:

- Administrador
- AnalistaOn
- AnalistaOff
- Designer
- Redator

Controlo de acesso baseado em roles

O sistema implementa controlo de acessos utilizando atributos declarativos [Authorize], permitindo restringir o acesso a determinadas páginas e ações com base no cargo do utilizador.

Exemplo:

```
[Authorize(Roles = "Administrador")]
```

Esta abordagem garante que apenas utilizadores autorizados possam aceder a funcionalidades como gestão de utilizadores, clientes e cargos.

Adicionalmente, foram implementadas verificações na camada de apresentação (views), utilizando expressões como:

User.IsInRole("Administrador")

para controlar a visibilidade de botões e ações e reforçar a segurança da aplicação.

Integração com o domínio da aplicação

O sistema Identity foi integrado com as entidades principais do AdFlow, nomeadamente:

- Campanhas
- Materiais
- Fluxos de campanha

Cada registo destas entidades mantém uma referência ao utilizador que o criou (FuncionarioId), permitindo rastreabilidade e auditoria das ações realizadas no sistema.